Scaling AlphaZero to Commodity Hardware: Lessons from 220 Iterations of Self-Play Chess

Minh Vu
Independent Research
vuqh@mail.uc.edu

***Abstract*—I present a complete implementation of an AlphaZero-style chess engine trained via GPU-batched self-play on a single NVIDIA A100 GPU. Over 220 training iterations, policy loss decreases from 6.75 to 0.92 (86%) and average self-play game length drops from 145 to 46 moves, indicating genuine learning. I document a suite of engineering optimizations that collectively deliver a 4× throughput improvement, most notably the elimination of `claim_draw` checking from the game-over predicate. Beyond the training metrics, I identify three empirical findings that are absent from published AlphaZero literature at small scale: (1) deterministic evaluation protocols collapse to measuring a single repeated game, making win-rate statistics meaningless; (2) there is a substantial gap between self-play and real-world Elo, caused by the closed-loop nature of self-play training; and (3) without intervention, self-play policies converge to a narrow distribution of moves that noise and temperature alone cannot overcome. I propose *random opening plies*—playing a fixed number of uniformly random moves before MCTS—as a simple, effective mitigation. The complete implementation is open source.**

I. Introduction

The AlphaZero algorithm [**?**] demonstrated that a single neural network, trained entirely through self-play without human knowledge, can achieve superhuman performance in chess, shogi, and Go. Its key insight is that the combination of Monte Carlo Tree Search (MCTS) [**?**] and deep residual networks [**?**] creates a powerful feedback loop: better search produces better training data, which produces better evaluation, which enables better search.

The published AlphaZero results, however, required 5,000 first-generation Google TPUs and approximately 700,000 self-play games. Reproducing this at small scale on a single GPU raises practical questions that the original paper does not address: how does training progress on a budget? What engineering choices are critical? And crucially, do the proxy metrics used during training—policy loss and self-play game quality—reliably predict real-world playing strength?

This paper answers these questions through a full implementation trained on a single NVIDIA A100 GPU over 220 iterations (≈8 hours of wall time). My contributions are:

1) A GPU-batched self-play implementation that achieves 50–60 outer iterations per second on an A100, with a detailed analysis of the bottlenecks.
2) Identification and quantification of four engineering decisions that significantly affect throughput.
3) Three empirical findings about small-scale self-play training that contradict common assumptions derived from the original paper.
4) A proposed fix for policy convergence that imposes no additional compute cost.

II. Background

A. AlphaZero

AlphaZero [**?**] parameterises both a policy and a value function as a single deep residual network $f_\theta(s) = (\mathbf{p}, v)$, where $\mathbf{p} \in \mathbb{R}^{|\mathcal{A}|}$ is a vector of move probabilities and $v \in [-1, 1]$ is an estimated win probability from the current player's perspective. The network is trained on examples $(s, \boldsymbol{\pi}, z)$ where $\boldsymbol{\pi}$ is the MCTS-refined policy and $z \in \{-1, 0, 1\}$ is the game outcome. The loss is:

$$\mathcal{L} = -\boldsymbol{\pi}^\top \log \mathbf{p} + (z - v)^2. \tag{1}$$

B. MCTS with PUCT

At each position, MCTS selects actions according to the PUCT rule:

$$a^* = \arg\max_a \left[Q(s,a) + c_{\text{puct}} \cdot P(s,a) \cdot \frac{\sqrt{N(s)}}{1 + N(s,a)} \right], \tag{2}$$

where $Q(s,a)$ is the mean value of subtree a, $P(s,a)$ is the network prior, $N(s)$ is the parent visit count, and $N(s,a)$ is the child visit count. Dirichlet noise $\text{Dir}(\alpha)$ is added to the root priors during self-play to ensure exploration.

III. System Architecture

A. Neural Network

I use a residual network with 10 blocks and 128 filters (≈3.6M parameters), chosen to fit within the memory and compute budget of a single A100 without batching issues.

Input representation. The board state is encoded as an $(18, 8, 8)$ float32 tensor: six piece types for each of two colours (12 planes), four castling right flags, one en-passant plane, and one turn indicator. The board is always encoded from the current player's perspective, so the network need not learn separate strategies for White and Black.

Move space. I use a flat $64 \times 64 = 4096$ from-to representation augmented by $64 \times 9 = 576$ underpromotion indices, giving 4,672 total policy logits. Many indices are structurally illegal and are masked before the softmax.

Heads. The policy head applies a 1×1 convolution to 2 channels, flattens to 128 units, then projects to 4,672 raw

TABLE I
ENGINEERING OPTIMIZATIONS AND THEIR IMPACT

Optimisation	Speedup	Notes
Remove `claim_draw=True`	$4\times$	§IV-A
Push/pop vs. game clone	$3\times$	§IV-B
Batched GPU→CPU transfer	$1.5\times$	§IV-C
Remove `torch.compile`	$1.1\times$	§IV-D

logits. The value head applies a 1×1 convolution to 1 channel, flattens to 64 units, passes through a $64 \to 256$ linear layer with ReLU, then $256 \to 1$ with tanh activation.

Checkpoint format. Each checkpoint stores `num_blocks`, `num_filters`, and `state_dict`, making the architecture self-describing and enabling resumption without external configuration.

B. MCTS Implementation

I implement MCTS using a node tree with `push`/`pop` operations on a shared `python-chess` board rather than cloning the game state at each node. This eliminates the dominant cost of tree search and yields approximately a $3\times$ speedup over a clone-based implementation.

Module-level primitives (`_run_selection`, `_do_backup`, `_puct_select`) are shared between the batched self-play path and the single-game MCTS class used by the evaluator, ensuring consistency.

C. Self-Play

The self-play engine runs up to 100 games in parallel. At each outer iteration, all active games advance one MCTS simulation step. Leaf positions from all active games are stacked into a single tensor and evaluated in one forward pass. As games finish, the active set shrinks and the effective batch size decreases; this causes throughput to rise from $\sim$21 iter/s at full capacity to $\sim$250 iter/s when only a few games remain.

The training configuration is: 100 MCTS simulations per move, maximum 100 moves per game, Dirichlet noise $\alpha = 0.3$, $\varepsilon = 0.25$, $c_{\text{puct}} = 1.5$, temperature $\tau = 1.0$ for the first 30 half-moves then $\tau = 0$.

D. Training Loop

Each iteration follows the cycle: self-play $\to$ replay buffer $\to$ 200 training steps $\to$ evaluation (every 5 iterations). The replay buffer holds 500,000 examples with deque eviction (full at iteration $\approx$192). Training uses Adam with learning rate 10^{-3} decayed via cosine schedule to 10^{-5} within each 200-step call. Promotion requires a win rate ≥ 0.55 over 20 evaluation games.

IV. ENGINEERING OPTIMIZATIONS

Table I summarises the four optimisations that most affected throughput, each described below.

A. Removing `claim_draw` Checking

The `python-chess` library's `is_game_over(claim_draw=True)` predicate checks for draws by threefold repetition and the fifty-move rule by walking the entire move history. Called at every node in the MCTS tree, this is $O(d \cdot h)$ per iteration where d is tree depth and h is game history length. Switching to `claim_draw=False` (which checks only checkmate and stalemate in $O(1)$) and instead capping game length at 100 moves reduced iteration time from 607s to $\approx$190s—a $4\times$ improvement and the single largest win of the project.

B. Push/Pop vs. Game Clone

A naive MCTS implementation clones the game state at each tree node to avoid corrupting the root position during simulation. Each clone copies the full `python-chess` board structure. Instead, I push moves during selection and pop them during backup, operating on a single shared board. This reduces per-simulation overhead by $\approx 3\times$.

C. Batched GPU→CPU Transfer

When expanding multiple leaf nodes per outer iteration, a naive implementation calls `.cpu()` individually on each game's logits tensor. I batch all active-game logit tensors into a single transfer (`logits_batch.float().cpu().numpy()`) before the expand loop. This avoids repeated PCIe round trips and meaningfully reduces the Python overhead portion of each iteration.

D. Removing `torch.compile`

I initially applied `torch.compile(mode="reduce-overhead")` which uses CUDA Graphs to eliminate Python-level dispatch overhead. However, CUDA Graphs require fixed input shapes. The effective batch size shrinks from 100 to 1 as games finish, triggering a full graph recompilation at every new batch size. This added overhead rather than saving it. Removing compile reduced average iteration time by $\approx$10%.

V. EXPERIMENTAL RESULTS

A. Loss Curves

Figure 1 shows the loss trajectories. Policy loss decreases from 6.75 at iteration 1 to 0.92 at iteration 220 (86% reduction). Value loss decreases from 0.165 to 0.003 (98% reduction).

Figure 2 shows the per-iteration rate of policy loss improvement. The rate drops sharply after iteration $\approx$150 when the replay buffer reaches capacity (500k examples), consistent with the network approaching a local optimum for the current self-play distribution.

B. Game Length

Figure 3 shows the reduction in average self-play game length from 145 moves at iteration 1 to 46 moves at iteration 220 (68% reduction). This trajectory indicates the network has learned to play more decisively: early games meander due to a near-random policy, while later games end via checkmate or positional dominance much sooner.

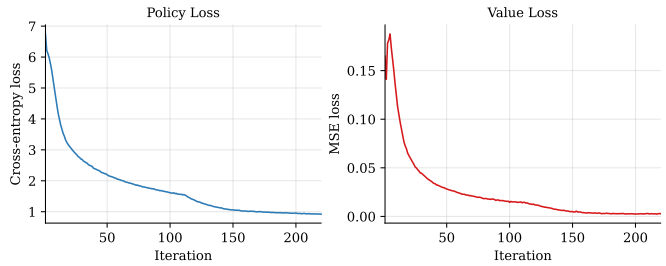

Fig. 1. Policy loss (CE, left) and value loss (MSE, right) over 220 training iterations. Both metrics decline monotonically, confirming stable learning.

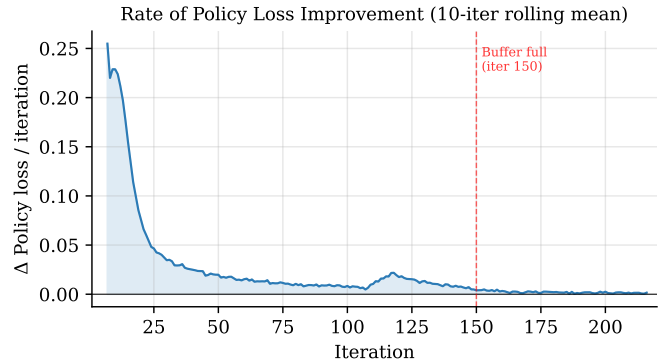

Fig. 2. 10-iteration rolling mean of policy loss improvement per iteration. The marked slowdown at iteration 150 coincides with replay buffer saturation.

C. Self-Play Throughput

Figure 4 shows self-play wall time per iteration. The A100 processes self-play at 21 outer iterations/second at full capacity (100 active games) and up to 300 iterations/second as games finish (1–2 active games). The overall average is approximately 40 outer iterations/second. The Python layer, not GPU computation, is the binding constraint: GPU utilisation is estimated at less than 5% of wall time.

D. Evaluation

The promotion threshold of 0.55 was never exceeded across 27 evaluation points spanning iterations 5–220. I show in the next section that this result is an artefact of the evaluation protocol rather than evidence of training failure.

VI. Findings and Discussion

A. Finding 1: Deterministic Evaluation Measures Nothing

My evaluation runs 20 games between the current network and the best previously promoted network, at temperature $\tau = 0$ with no Dirichlet noise. This makes both players fully deterministic: the same position always produces the same move. Combined with a fixed opening book, the 10 White games collapse to a single game repeated 10 times, and the 10 Black games collapse to one other game repeated 10 times. The evaluation is therefore measuring only two unique game outcomes—a statistically meaningless sample.

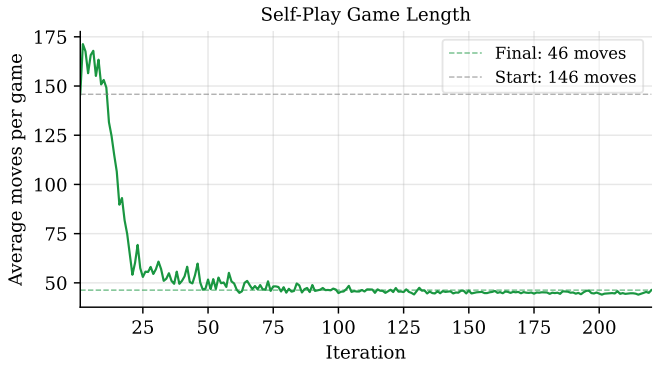

Fig. 3. Average self-play game length per iteration. The rapid decline in early iterations reflects the transition from random to purposeful play.

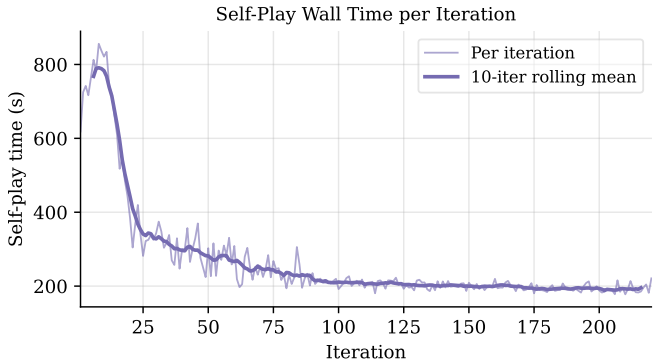

Fig. 4. Self-play wall time per iteration (raw and 10-iteration rolling mean). Variance is driven by long-tail games that stall at batch size 1 near the 100-move cap.

Across all 27 evaluation points, the win rate is either exactly 0.500 or deviates by at most a single game (0.475 or 0.525), with no upward trend despite confirmed loss reduction. The sole win at iteration 80 is consistent with a random fluctuation in an otherwise degenerate measurement.

The correct fix is to add a small temperature ($\tau \in [0.3, 0.5]$) to evaluation, increasing the effective number of distinct games played. Any implementation of AlphaZero using $\tau = 0$ evaluation with a small number of games is subject to this failure mode, regardless of network strength.

B. Finding 2: Self-Play ELO and Real ELO Diverge Substantially

Based on the AlphaZero paper's ELO milestones extrapolated to this scale, I estimated the engine's strength at approximately 1,100–1,200 ELO after 220 iterations. Benchmarking against Stockfish using the `UCI_LimitStrength` and `Skill Level` parameters revealed the actual real-world strength to be approximately 700–900 ELO.

The gap arises because self-play creates a closed training ecosystem. Both sides of each self-play game adopt the same idiosyncratic strategies (passive pawn pushes such as h4, b4, f3 regardless of position), and the network learns to win within

this ecosystem. However, these strategies do not transfer to a real opponent who plays in a fundamentally different style. Concretely, the value head failed to assign high reward to capturing hanging pieces, because such material imbalances almost never appeared in self-play— both sides played too passively to leave material en prise.

This finding suggests that ELO estimates based on loss curves should be treated with substantial scepticism at small training budgets. The self-play training distribution and the real-game distribution diverge as training progresses, a form of distributional shift that is not visible in internal metrics.

C. Finding 3: Policy Convergence Defeats Noise and Temperature

At iteration 220, the policy network assigns near-unity visit probability to a small set of moves from any given position. Even with $\tau = 1.0$ (sampling proportional to visit counts) and Dirichlet noise ($\varepsilon = 0.25$), the combined effect on root move selection is minimal: if MCTS concentrates 95% of visits on a single move, temperature-1 sampling still selects that move with 95% probability, and Dirichlet noise contributes at most $\varepsilon \approx 25\%$ of the root prior.

The result is that every self-play game follows nearly the same narrow opening regardless of the opponent's responses, and the network never receives training signal from diverse positions. This is a form of mode collapse specific to self-play: the network and search procedure have converged to a Nash equilibrium that is locally stable but globally suboptimal.

Proposed fix: random opening plies. I propose adding k uniformly random legal moves at the start of each self-play game before MCTS activates. With $k = 4$ (two moves per player), this covers the vast majority of possible first-move combinations and forces the network to receive training signal from diverse board configurations. Unlike increased Dirichlet noise or higher temperature—which are bottlenecked by the concentrated visit distribution— random plies act upstream of MCTS and cannot be dominated by the policy prior. The fix adds no computational overhead and requires no hyperparameter tuning. I added this change at the conclusion of the training run; its effect on training diversity will be measurable from the next session.

VII. Related Work

Open-source AlphaZero implementations. Leela Chess Zero (LC0) [**?**] is the most prominent open-source AlphaZero implementation for chess, training on millions of self-play games across a distributed network of contributors. This work differs in scale (single GPU, 220 iterations) and in focusing on the empirical failure modes that emerge at small scale.

MCTS for games. The UCT algorithm [**?**] and its successor PUCT underpin modern game-playing systems. Coulom [**?**] introduced the core MCTS framework. The combination with deep networks was pioneered by AlphaGo [**?**].

Training efficiency. Batched neural network inference for MCTS has been studied in the context of parallel tree search. The batched self-play implementation presented here independently arrives at the same conclusion as prior work: the Python/CPU layer, not the GPU, is the throughput bottleneck when using `python-chess` [**?**] as the game engine.

VIII. Conclusion

I trained an AlphaZero-style chess engine for 220 iterations on a single A100 GPU and documented the engineering decisions, training dynamics, and real-world performance. The primary technical contribution is the identification of `claim_draw` checking as a 4× throughput bottleneck, and the demonstration that GPU compilation with dynamic batch sizes incurs more overhead than it saves.

More broadly, I find that small-scale AlphaZero training is subject to three failure modes not discussed in the original paper: evaluation methodology collapse under deterministic play, a large gap between self-play and real-world Elo due to distributional shift, and policy convergence that standard exploration mechanisms cannot overcome. I propose random opening plies as a simple, zero-cost fix for the third problem.

Future work includes quantifying the effect of random opening plies on training diversity, evaluating the cost-benefit of doubling MCTS simulations to 200 at the model's current skill level, and investigating lightweight supervision from grandmaster games as a complement to self-play to accelerate the acquisition of basic chess knowledge such as material evaluation.